



A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

UNIVERSITÀ DEGLI STUDI DI PERUGIA
DIPARTIMENTO DI INGEGNERIA

Tesina Finale di
Algoritmi e Strutture Dati

Corso di Laurea in Ingegneria Informatica ed Elettronica — A.A. 2025–2026

**Scrittura di un
algoritmo per la
generazione di labirinti**

Docente
Prof. Emilio Di Giacomo

Studente
Romeo Miscio
Matricola: 364672

Ultimo aggiornamento: 13 luglio 2026

Indice

1	Il problema della generazione di labirinti	1
2	Modellazione formale tramite Teoria dei Grafi	1
3	Obiettivi del progetto	2
4	Algoritmi implementati	2
4.1	Randomized Depth-First Search (Recursive Backtracker)	2
4.1.1	Scelte implementative e gestione dello Stack	3
4.1.2	Analisi della complessità teorica	4
4.2	Algoritmo di Kruskal Randomizzato	4
4.2.1	La struttura dati Disjoint-Set (Union-Find)	4
4.2.2	Analisi della complessità teorica	5
4.2.3	Funzione di iterazione di Ackermann	5
4.3	Algoritmo di Prim modificato	5
4.3.1	Scelte implementative sulla frontiera	5
4.3.2	Analisi della complessità teorica	6
4.4	Algoritmi basati su Random Walk	6
4.4.1	Algoritmo di Aldous-Broder	6
4.4.2	Analisi della complessità teorica	7
4.4.3	Algoritmo di Wilson	7
4.4.4	Analisi della complessità teorica	8
4.5	Algoritmi geometrici e a partizione spaziale	8
4.5.1	Algoritmo di Divisione Ricorsiva	8
4.5.2	Analisi della complessità teorica	9
4.5.3	Algoritmo di Eller	9
4.5.4	Analisi della complessità teorica	10
5	Esperimenti e analisi dati	10
5.1	Setup sperimentale	10
5.1.1	Metodologia di profilazione e campionamento	10
5.2	Risultati sperimentali globali	11
5.3	Discussione critica dei risultati	11
5.3.1	Il dominio degli approcci Top-Down: Divisione Ricorsiva	12
5.3.2	L'efficienza dei Graph Traversal e delle strutture dati: DFS, Prim, Kruskal ed Eller	12
5.3.3	Analisi comparativa dei paradigmi stocastici: Wilson e Aldous-Broder	13

1 Il problema della generazione di labirinti

Il problema della generazione di labirinti perfetti consiste nel ricavare una struttura planare complessa dotata di un'unica interconnessione spaziale priva di interruzioni (regioni isolate) e priva di percorsi alternativi chiusi (cicli) [1].

In ambito informatico e computazionale, un labirinto non rappresenta semplicemente un elemento ludico, ma si configura come un eccellente banco di prova per lo studio della topologia strutturale, della teoria dei grafi e dell'efficienza algoritmica nell'esplorazione spaziale. Trova inoltre ampie applicazioni nella generazione procedurale di contenuti (PCG) nei videogiochi, nella pianificazione di percorsi per la robotica mobile e nella simulazione di reti di micro-canali biologici o idraulici.

2 Modellazione formale tramite Teoria dei Grafi

Il modo formalmente più rigoroso per descrivere la topologia di un labirinto adotta le definizioni della teoria dei grafi. Una griglia bidimensionale predeterminata (sia essa rettangolare, esagonale o circolare) può essere mappata direttamente su un grafo non diretto e pesato $G = (V, E)$, in cui:

- L'insieme dei vertici V rappresenta l'insieme delle celle che compongono lo spazio calpestabile del labirinto, con $|V| = n$;
- L'insieme degli archi E rappresenta l'insieme di tutti i passaggi originariamente potenziali (ovvero l'assenza di un muro divisorio) tra celle geometricamente adiacenti. Inizialmente non esistono muri eretti.

Generare un labirinto strutturalmente valido significa individuare un sottografo co-prente $G' = (V, E')$ tale che $G' \subseteq G$, che rispetti due vincoli stringenti:

1. **Connettività:** Il sottografo G' deve essere strettamente connesso. Se G' non fosse connesso, esisterebbero regioni isolate e strutturalmente irraggiungibili all'interno della griglia, invalidando l'integrità del labirinto.
2. **Alinearità (Assenza di cicli):** Il sottografo G' deve essere aciclico. La presenza di un ciclo implicherebbe l'esistenza di cammini multipli per connettere due nodi qualsiasi, riducendo la complessità intrinseca del labirinto e violando la definizione classica di "labirinto perfetto".

Dalle proprietà fondamentali della teoria dei grafi, un sottografo di G che contiene tutti i vertici V , che sia connesso e che è privo di cicli, per definizione è un **Albero di Copertura** (o *Spanning Tree*). Di conseguenza, l'intero problema della generazione procedurale di labirinti si riduce all'estrazione di uno **Spanning Tree casuale** partendo dal grafo iniziale della griglia. I muri del labirinto finito saranno costituiti esattamente dal complemento degli archi selezionati, ovvero dall'insieme degli archi esclusi $E \setminus E'$.

3 Obiettivi del progetto

L'obiettivo principale di questo progetto è implementare, analizzare e confrontare sperimentalmente le prestazioni e le caratteristiche topologiche delle principali famiglie di algoritmi per l'estrazione di Spanning Tree casuali.

Nello specifico, il software sviluppato in linguaggio Java mira a ricoprire l'intero spettro algoritmico descritto nella pagina di Wikipedia [1], dividendoli per tipologia di funzionamento:

- **Algoritmi basati su Tree/Graph Traversal:** Randomized Depth-First Search (Recursive Backtracker);
- **Algoritmi basati su Minimum Spanning Tree adattati:** Algoritmo di Kruskal randomizzato e Algoritmo di Prim modificato.
- **Algoritmi basati su Random Walk:** Algoritmo di Aldous-Broder e Algoritmo di Wilson.
- **Algoritmi geometrici o a divisione spaziale:** Algoritmo di Divisione Ricorsiva ed Eller's Algorithm.

Oltre all'analisi prestazionale pura asintotica e di wall-clock (tempo di esecuzione del programma), il progetto si propone di integrare un motore grafico per la visualizzazione dinamica passo-passo della crescita dello Spanning Tree. Al fine di garantire un'analisi visiva coerente ed equa tra i diversi paradigmi, il sistema adotta un intervallo di campionamento temporale costante per singola azione atomica ($delay = 17\ ms$). Questa scelta implementativa si rivela di fondamentale importanza: poiché il numero di operazioni necessarie alla creazione del labirinto varia drasticamente tra gli approcci deterministici (es. DFS) e quelli stocastici a passeggiata casuale (es. Aldous-Broder), un delay fisso permette all'osservatore di valutare visivamente e in tempo reale l'effettiva densità operativa e l'efficienza strutturale di ciascun algoritmo direttamente sul rendering della griglia.

4 Algoritmi implementati

In questo capitolo vengono analizzati in dettaglio tutti e sette gli algoritmi sviluppati per la generazione di labirinti perfetti. Ciascun approccio interpreta il problema dell'estrazione di uno Spanning Tree casuale sfruttando strutture dati e metodi differenti.

4.1 Randomized Depth-First Search (Recursive Backtracker)

L'algoritmo basato sulla ricerca in profondità (DFS) modificata è un approccio classico di tipo *backtracking*. Esso tende a generare labirinti caratterizzati da un basso numero di diramazioni corte e da corridoi marcatamente lunghi e tortuosi (frequenti vicoli ciechi profondi), proprietà topologica nota in letteratura come "alto fattore di fiumi".

4.1.1 Scelte implementative e gestione dello Stack

La formulazione puramente ricorsiva dell'algoritmo DFS delega implicitamente il tracciamento del cammino di *backtracking* allo stack dei *frame* di attivazione della Java Virtual Machine. Ciascuna chiamata nidificata alloca un record di memoria isolato in un'area ad accesso rapido ma strutturalmente limitata (tipicamente configurata di default a un massimo di 1 MB per thread). Nello scenario computazionale peggiore — rappresentato da una topologia di labirinto lineare o a serpentina — la profondità della ricorsione scala linearmente con il numero totale di nodi ($n = |V|$), esaurendo rapidamente lo spazio allocabile e provocando il crash irreversibile dell'applicazione tramite un'eccezione di *StackOverflowError*.

Per ovviare a questo vincolo hardware, il motore di generazione è stato convertito in una variante iterativa accoppiata a una **vettorializzazione esplicita dello stack**. Tale tecnica prevede la rimozione delle chiamate a funzione a favore dell'uso diretto della struttura dati `java.util.Stack`. Dal punto di vista architetturale, questa scelta sposta il meccanismo di memorizzazione dei nodi dallo Stack di esecuzione alla memoria *Heap* globale, la cui dimensione nel nostro setup sperimentale è protetta ed estesa fino a 2 GB tramite i flag di sistema `-Xms2G -Xmx2G`.

Il controllo del flusso operativo viene così sottratto all'automazione rigida della JVM e governato esplicitamente nel codice sorgente mediante le istruzioni atomiche **push** (inserimento) e **pop** (estrazione). Poiché l'espansione geometrica dell'array sottostante garantisce un costo computazionale pari a $\mathcal{O}(1)$ ammortizzato per singola operazione, l'approccio iterativo assicura uno *scaling* sicuro e matematicamente illimitato, eliminando alla radice il rischio di saturazione della memoria anche su domini metrici di scala macroscopica.

Il comportamento del singolo passo operativo è regolato dalla seguente logica a stati stabili:

1. **Fase di inizializzazione (Seeding):** Prima dell'avvio del ciclo iterativo, lo stack si presenta vuoto. L'algoritmo seleziona una cella di partenza arbitraria all'interno del dominio della griglia (tipicamente il vertice di coordinate $(0,0)$), la contrassegna formalmente nello stato di "non visitata" e vi esegue sopra un'operazione di **push()**. Questa cella costituisce il seme iniziale e la radice geometrica dell'intero albero di copertura.
2. **Ispezione dell'apice (The Peek Phase):** All'inizio di ogni iterazione del ciclo principale (il quale prosegue fintanto che lo stack non è vuoto), il sistema interroga l'elemento in cima alla struttura dati mediante il metodo **peek()**, senza tuttavia rimuoverlo. Questa cella rappresenta la testa corrente del cammino di esplorazione attiva.
3. **Campionamento stocastico dei vicini:** L'algoritmo esamina i quattro punti cardinali adiacenti alla cella estratta per identificare i vicini geometrici che si trovano ancora nello stato di "non visitati". Se tale insieme non è vuoto:
 - Viene estratto un vicino in modo equiprobabile sfruttando un generatore di numeri pseudo-casuali (approccio *Randomized*).
 - Si procede all'abbattimento fisico delle barriere divisorie corrispondenti nella matrice dei muri, inserendo l'arco nell'insieme E' .

- La cella vicina viene marcata come “visitata” per impedirne il ricampionamento futuro.
 - La cella viene immediatamente inserita in cima allo stack tramite un’istruzione di `push()`, diventando a tutti gli effetti la nuova testa del cammino per l’iterazione successiva.
4. **Fase di Backtracking (The Pop Phase):** Qualora l’ispezione dei vicini determini che tutte le celle adiacenti alla testa attuale del cammino sono già state visitate (raggiungimento di un vicolo cieco locale), l’algoritmo attiva la routine di arretramento. La cella in esame viene definitivamente rimossa dallo stack mediante l’operazione di `pop()`. Alla successiva iterazione, l’istruzione `peek()` esaminerà la cella precedente, ripercorrendo a ritroso il corridoio fino a intercettare una biforcazione con vicini ancora inesplorati.

4.1.2 Analisi della complessità teorica

In una griglia regolare planare, il grado massimo di ogni vertice è limitato superiormente da una costante ($\deg(v) \leq 4$). Di conseguenza, il numero totale di archi è lineare rispetto al numero dei nodi, ovvero $|E| = \mathcal{O}(V)$. Poiché ogni cella viene inserita ed estratta dallo stack esattamente una volta, il ciclo interno esegue al più $2|V|$ iterazioni. L’accesso e la modifica delle proprietà della cella (muri e flag di visita) avvengono in tempo costante $\mathcal{O}(1)$. La complessità temporale asintotica dell’algoritmo è pertanto strettamente lineare:

$$\mathcal{O}(V)$$

Lo spazio aggiuntivo occupato dallo stack nel caso pessimo (un unico corridoio lineare senza diramazioni) coincide con l’altezza massima dell’albero, richiedendo una complessità spaziale pari a $\mathcal{O}(V)$.

4.2 Algoritmo di Kruskal Randomizzato

L’adattamento dell’algoritmo di Kruskal per la generazione di labirinti opera trattando tutti i muri divisori della griglia come un insieme di archi non pesati. Al posto di minimizzare il costo totale ordinando gli archi, l’algoritmo estrae un arco in modo puramente casuale a ogni iterazione, fondendo progressivamente i sotto-alberi isolati.

4.2.1 La struttura dati Disjoint-Set (Union-Find)

Per controllare in tempo quasi-costante se l’abbattimento di un muro introduce un ciclo indesiderato, è stata implementata da zero una foresta di insiemi disgiunti (**Disjoint-Set**) dotata di due ottimizzazioni fondamentali:

- **Unione per rango:** Attacca sempre l’albero con altezza minore sotto la radice dell’albero con altezza maggiore, limitando la crescita logaritmica delle strutture.
- **Compressione dei cammini:** Durante la fase di ricerca (`find`), ogni nodo visitato viene agganciato direttamente alla radice dell’insieme, appiattendolo la struttura per le invocazioni successive.

4.2.2 Analisi della complessità teorica

Inizialmente la lista contiene la totalità degli archi possibili della griglia, pari a $|E| \approx 2|V|$. Il mescolamento iniziale (`Collections.shuffle`) richiede tempo lineare $\mathcal{O}(E)$. A ogni passo estrattivo, l'algoritmo esegue due operazioni di `find` e al più una operazione di `union`. Sotto le ipotesi di compressione dei cammini e unione per rango, la complessità ammortizzata di ciascuna operazione è limitata superiormente dalla funzione inversa di una crescita esponenziale raddoppiata, espressa mediante la funzione di iterazione di Ackermann $\alpha(n)$. La complessità temporale complessiva per elaborare tutti gli archi è:

$$\mathcal{O}(E \cdot \alpha(V))$$

Poiché nella nostra griglia $|E| = \mathcal{O}(V)$ e la funzione α assume un valore pratico minore di 5 per qualsiasi input realistico, l'algoritmo opera in tempo di *wall-clock* quasi-lineare. Lo spazio aggiuntivo per mantenere i vettori dei padri e dei ranghi è pari a $\mathcal{O}(V)$.

4.2.3 Funzione di iterazione di Ackermann

L'efficienza macroscopica del paradigma di Kruskal risiede nell'ottimizzazione formale della struttura dati *Disjoint-Set* (Union-Find), deputata al rilevamento istantaneo dei cicli all'interno della foresta geometrica. Attraverso l'azione congiunta delle euristiche di bilanciamento per rango (*union by rank*) e di compressione dei cammini (*path compression*), il costo computazionale richiesto per eseguire le operazioni di ricerca (`find`) e fusione (`union`) subisce un abbattimento asintotico radicale.

Dal punto di vista analitico, la complessità temporale ammortizzata per singola operazione si attesta a $\mathcal{O}(\alpha(V))$, dove α rappresenta la funzione inversa di Ackermann. Poiché la funzione originaria di Ackermann esibisce una crescita iper-esplosiva basata su torri di potenze e tetrazioni, la sua controparte inversa $\alpha(V)$ delinea una progressione iper-lenta, assumendo un valore strettamente inferiore a 4 per qualsiasi input fisicamente concepibile nel mondo reale (inclusi domini infinitamente superiori alla totalità degli atomi nell'universo osservabile). Questo legame matematico consente di decretare che, nell'ambito dell'architettura software sviluppata, le operazioni di gestione degli insiemi disgiunti avvengono in tempo quasi-costante $\mathcal{O}(1)$ a runtime, giustificando rigorosamente la stabilità e la fluidità di calcolo registrate in fase di profilazione sperimentale.

4.3 Algoritmo di Prim modificato

L'algoritmo di Prim si sviluppa espandendo radialmente un unico sotto-albero a partire da una cella di innesco casuale. Mantiene costantemente traccia di una struttura di "frontiera", definita come l'insieme di tutti i muri che separano una cella già inclusa nel labirinto da una cella non ancora esplorata.

4.3.1 Scelte implementative sulla frontiera

Nell'implementazione didattica standard dell'algoritmo di Prim (utilizzato per la ricerca del Minimum Spanning Tree classico), la frontiera viene memorizzata in una coda a priorità (*Min-Heap*) basata sul peso degli archi. Nel contesto della generazione procedurale

casuale, l'assenza di pesi deterministici ci permette di sostituire lo heap con una lista dinamica indicizzata (`java.util.ArrayList`).

La rimozione di un elemento casuale da una lista basata su array richiede lo spostamento degli elementi successivi, operazione intrinsecamente inefficiente se eseguita al centro della struttura ($\mathcal{O}(N)$). Per mantenere il costo microscopico nell'interfaccia a passi, si memorizzano i riferimenti strutturali espliciti dei muri (`PrimWall`) e si effettua un campionamento ad accesso diretto.

4.3.2 Analisi della complessità teorica

Ogni cella viene visitata una sola volta e aggiunge al più 4 muri alla lista di frontiera. Di conseguenza, il numero massimo di elementi immessi nella lista è proporzionale a $4|V|$. L'estrazione casuale di un indice da una struttura `ArrayList` avviene in tempo costante $\mathcal{O}(1)$. Per evitare l'overhead di rimozione standard (che comporterebbe uno slittamento lineare degli elementi pari a $\mathcal{O}(V)$), l'implementazione adotta una tecnica di ottimizzazione in tempo costante: l'elemento selezionato viene scambiato in posizione con l'ultimo elemento della lista, e quest'ultimo viene rimosso tramite `remove(list.size() - 1)`. Poiché l'ordine della frontiera non influisce sulla natura stocastica dell'algoritmo, quest'operazione azzerà l'overhead di shifting portando il costo di ogni singola rimozione a $\mathcal{O}(1)$. La complessità temporale complessiva su griglia si attesta quindi a:

$$\mathcal{O}(V)$$

Dal punto di vista spaziale, la frontiera immagazzina al più il perimetro dinamico di crescita dell'albero, occupando in memoria uno spazio delimitato superiormente da $\mathcal{O}(V)$.

4.4 Algoritmi basati su Random Walk

La generazione di un labirinto tramite passeggiate casuali (*Random Walk*) si fonda su principi profondi della teoria delle probabilità e dei processi markoviani. Entrambi gli algoritmi descritti di seguito condividono la proprietà matematica di estrarre un albero di copertura da una distribuzione perfettamente uniforme (*Uniform Spanning Tree*), garantendo che ogni possibile labirinto perfetto compatibile con la griglia iniziale abbia l'esatta stessa probabilità di essere generato.

4.4.1 Algoritmo di Aldous-Broder

L'algoritmo di Aldous-Broder rappresenta uno dei primi e più celebri approcci stocastici per la generazione di Spanning Tree uniformi su grafi generici. Il suo funzionamento si basa sul concetto puro di passeggiata casuale (*Random Walk*) e non richiede alcuna pianificazione geometrica o vincolo topologico preventivo.

Il flusso esecutivo del paradigma può essere sintetizzato nei seguenti passaggi logici sequenziali:

1. Si seleziona una cella di partenza in modo completamente casuale all'interno della griglia e la si marca nello stato del sistema come "visitata".

2. Fintanto che il numero complessivo di celle visitate è strettamente inferiore al volume totale dei vertici del grafo ($|V|$), l'algoritmo sceglie ad ogni iterazione un vicino adiacente in modo equiprobabile e si sposta su di esso.
3. Se il vicino selezionato non è ancora stato esplorato dall'inizio della simulazione, l'arco che connette la cella corrente a quest'ultimo viene ufficialmente inserito all'interno dello Spanning Tree (E'). Dal punto di vista applicativo, questa operazione comporta l'abbattimento immediato del muro divisorio corrispondente. Successivamente, la nuova cella viene marcata come visitata.
4. Se il vicino risulta invece già visitato, la passeggiata casuale si sposta semplicemente su di esso senza alterare la struttura dei confini e senza introdurre alcun arco nel grafo, al fine di scongiurare la formazione di cicli.

4.4.2 Analisi della complessità teorica

A differenza degli approcci deterministici, il tempo di esecuzione del paradigma di Aldous-Broder è strettamente legato al concetto statistico di *cover time* (tempo di copertura) di una passeggiata casuale. Sebbene su topologie lineari o grafi a manubrio (*barbell graphs*) il tempo di convergenza possa degenerare a $\mathcal{O}(V^2)$, la letteratura matematica dimostra che per un grafo a griglia bidimensionale regolare (2D Grid Graph) il valore atteso del tempo di copertura si attesta a $\frac{2}{\pi}V \ln^2 V$. Pertanto, la complessità temporale media dell'algoritmo applicato alla generazione di labirinti standard risulta essere pari a:

$$\mathcal{O}(V \log^2 V)$$

Dal punto di vista dello spazio ausiliario, l'algoritmo si rivela estremamente snello: non necessitando di strutture di tracciamento dinamico dei cammini o di stack ricorsivi di sistema, richiede esclusivamente una mappa booleana dei nodi visitati proporzionale al volume dei vertici, determinando un'occupazione spaziale fissa pari a:

$$\mathcal{O}(V)$$

4.4.3 Algoritmo di Wilson

L'algoritmo di Wilson introduce un'ottimizzazione radicale nel panorama della generazione stocastica, garantendo la creazione di uno Spanning Tree perfettamente uniforme in tempi asintoticamente inferiori rispetto al modello di Aldous-Broder. Il nucleo concettuale dell'algoritmo risiede nelle passeggiate casuali con cancellazione dei cicli (*Loop-Erased Random Walks - LERW*).

L'esecuzione si articola secondo la seguente transizione di stati operativi:

1. Si seleziona una cella arbitraria della griglia e la si inserisce nel labirinto stabilizzato. Questo singolo nodo costituisce la radice iniziale dello Spanning Tree parziale.
2. Si sceglie una qualsiasi cella non ancora appartenente al labirinto consolidato e da essa si fa partire una passeggiata casuale (*Random Walk*).

3. Durante il cammino, se la traiettoria interseca se stessa formando un cappio geometrico (ovvero entra in una cella già attraversata nella passeggiata corrente), l'algoritmo effettua un'operazione di *loop-erasure*: elimina istantaneamente il ciclo dalla memoria e riprende l'esplorazione stocastica direttamente dal punto di intersezione.
4. La passeggiata prosegue nel dominio finché non collide con una cella già integrata nel labirinto stabilizzato. A questo punto, l'intera traiettoria depurata dai cicli viene iniettata nella struttura permanente: tutti i suoi archi entrano a far parte dello Spanning Tree (E') e i relativi muri interni vengono abbattuti.
5. Il processo viene reiterato selezionando nuove celle di partenza esterne fino a quando ogni singolo vertice del grafo è stato mappato.

4.4.4 Analisi della complessità teorica

Dal punto di vista teorico, l'algoritmo di Wilson ottimizza la ricerca stocastica riducendo drasticamente i tempi morti associati alla convergenza nelle fasi finali della generazione. Su un grafo a griglia bidimensionale regolare (2D Grid Graph), sebbene il valore atteso del tempo di copertura asintotico appartenga alla stessa classe di Aldous-Broder, l'intersezione statistica tra la passeggiata corrente e l'albero parziale già eretto consente di eludere l'effetto rallentamento tipico del paradosso del collezionista di coupon. La complessità temporale media associata alla generazione completa si attesta matematicamente a:

$$O(V \log^2 V)$$

La complessità spaziale risente della necessità di mantenere in memoria la traiettoria temporanea della passeggiata prima del consolidamento.

4.5 Algoritmi geometrici e a partizione spaziale

A differenza degli approcci basati sui grafi che operano a livello locale di vertice o arco, gli algoritmi geometrici sfruttano la regolarità strutturale della griglia per partizionare lo spazio o per calcolare l'albero di copertura secondo vincoli di riga globali.

4.5.1 Algoritmo di Divisione Ricorsiva

L'algoritmo di Divisione Ricorsiva adotta una filosofia progettuale di tipo *top-down* guidata dal paradigma del *divide et impera*. Il processo si avvia considerando la griglia come un'unica macro-regione interamente priva di barriere interne.

L'esecuzione del singolo passo d'avanzamento si articola sulle seguenti sotto-fasi:

1. Viene estratta una regione geometrica dal buffer di elaborazione esplicito. Se le sue dimensioni in pixel logici di cella sono inferiori alla soglia critica di divisione ($w < 2$ o $h < 2$), la regione viene considerata atomica e consolidata.
2. Si valuta l'orientamento ottimale del taglio basandosi sul rapporto d'aspetto della sotto-matrice: se l'altezza supera la larghezza il taglio sarà preferenzialmente orizzontale, viceversa sarà verticale.

3. Viene eretto un muro continuo che seziona longitudinalmente la regione a una coordinata casuale. Successivamente, viene selezionata in modo uniforme una singola cella lungo la barriera appena eretta in cui praticare un'apertura (foro di transito).
4. La separazione genera due nuove sotto-regioni indipendenti, le quali vengono inserite nello stack per essere processate nelle iterazioni successive.

4.5.2 Analisi della complessità teorica

La scomposizione dello spazio ricalca in modo duale la struttura di un albero binario di decisione. Esprimendo la ricorrenza in funzione del numero complessivo di celle V , ad ogni livello della ricorsione l'algoritmo divide l'istanza corrente in due sotto-regioni distinte di dimensione pari a $V/2$. Il costo energetico richiesto per erigere la barriera divisoria è proporzionale al lato della sotto-griglia, attestandosi a $\mathcal{O}(\sqrt{V})$. Si delinea così la seguente relazione di ricorrenza fondamentale:

$$T(V) = 2T\left(\frac{V}{2}\right) + \mathcal{O}(\sqrt{V})$$

Applicando il Teorema dell'Esperto, i parametri strutturali risultano $a = 2$ e $b = 2$, definendo un valore critico di crescita delle foglie pari a $V^{\log_2 2} = V^1$. Poiché il driver asintotico locale $f(V) = \mathcal{O}(V^{0.5})$ è polilogaritmicamente inferiore al valore critico ($0.5 < 1$), l'istanza ricade strettamente nel Caso 1 del teorema. La complessità complessiva è dunque interamente dominata dal numero di nodi foglia, determinando un tempo di calcolo asintotico linearmente puro:

$$\mathcal{O}(V)$$

Dal punto di vista spaziale, la memoria ausiliaria necessaria per alloggiare i record geometrici delle regioni all'interno dello stack rispecchia l'altezza massima dell'albero di partizione, che si attesta in ogni scenario a un valore logaritmico pari a $\mathcal{O}(\log V)$.

4.5.3 Algoritmo di Eller

L'algoritmo di Eller rappresenta uno dei vertici più efficienti della generazione procedurale. Esso risolve il problema calcolando l'albero di copertura riga per riga in un unico passaggio sequenziale. Ciascuna cella della riga corrente viene associata a un identificativo numerico di set, il quale mappa in tempo reale la connettività globale del labirinto calcolato fino a quel momento.

La logica si sviluppa attraverso due macro-fasi per ogni riga:

1. **Fase Orizzontale:** L'algoritmo esamina ogni coppia di celle adiacenti sulla riga. Se appartengono a set diversi, decide in modo stocastico se abbattere il muro che le separa. In caso di abbattimento, i due set vengono fusi, assegnando a tutti i loro membri il medesimo identificativo. All'ultima riga del labirinto, questa unione viene forzata per ogni coppia avente set disgiunti, garantendo la connettività finale.
2. **Fase Verticale:** Per ciascun set distinto isolato sulla riga corrente, l'algoritmo seleziona casualmente almeno una cella (e potenzialmente più di una) e ne abbatte il muro inferiore (Sud), proiettando la connettività del set sulla riga sottostante. Le celle della riga successiva che non ricevono una connessione verticale vengono inizializzate con un identificativo di set completamente nuovo e vergine.

4.5.4 Analisi della complessità teorica

L'aspetto più straordinario dell'algoritmo di Eller risiede nella sua efficienza computazionale e spaziale. Per elaborare una singola riga di larghezza W , l'algoritmo esegue scansioni lineari sui vettori di stato dei set, impiegando un tempo costante per cella. Di conseguenza, l'intera griglia viene completata eseguendo un'unica passata fissa. La complessità temporale è strettamente lineare:

$$\mathcal{O}(V)$$

Dal punto di vista della memoria ausiliaria, l'algoritmo non necessita della conoscenza strutturale delle righe passate né di quelle future. Lo spazio richiesto è unicamente quello necessario a immagazzinare il vettore dei set della riga corrente, portando la complessità spaziale a un valore ottimale pari a $\mathcal{O}(\sqrt{V})$ (ovvero lineare rispetto alla sola larghezza della griglia W), rendendolo l'algoritmo ideale per la generazione di labirinti di dimensioni virtualmente infinite.

5 Esperimenti e analisi dati

5.1 Setup sperimentale

Al fine di garantire la riproducibilità metodologica e di isolare l'efficienza algoritmica pura dalle interferenze di sistema, le misurazioni prestazionali sono state condotte all'interno di un ambiente hardware e software standardizzato, descritto nei seguenti punti:

- **Processore:** CPU AMD Ryzen 7 5700G con grafica integrata Radeon Graphics (architettura x86_64, 8 core, 16 thread);
- **Memoria RAM:** 32 GB di memoria ad accesso casuale DDR4 operante ad alta frequenza (3600 MHz). Per azzerare l'overhead e le latenze parassite legate al ridimensionamento dinamico dello heap memory da parte della Java Virtual Machine, lo spazio runtime è stato bloccato a un'allocazione simmetrica iniziale e massima impostando i flag nativi `-Xms2G -Xmx2G`.
- **Ambiente software:** Sistema operativo Microsoft Windows 11, accoppiato all'ambiente di esecuzione ufficiale Oracle Java Run-Time Environment (JRE) versione 21.0.10 LTS. Il processo di profilazione e compilazione a runtime sfrutta l'architettura HotSpot™ 64-Bit Server VM (build 21.0.10+8-LTS-217), attiva in *mixed mode* con ottimizzazioni JIT (Just-In-Time) di livello C2 e condivisione dei dati di classe (*sharing*).

5.1.1 Metodologia di profilazione e campionamento

Come evidenziato nella parte legata al benchmarking su macchine virtuali regolate da Garbage Collection, la misurazione del tempo di esecuzione (*wall-clock time*) produce dati affetti da un alto coefficiente di varianza statistica. Per neutralizzare tali anomalie architetturali, la raccolta dati è stata strutturata in due fasi sequenziali distinte:

1. **Verifica strutturale preventiva:** Ogni algoritmo esegue una generazione preliminare su una griglia di test. Un modulo di controllo analitico verifica che il grafo risultante costituisca un vero albero di copertura privo di partizioni sconnesse, validando l'equazione di Eulero per i grafi planari ($|E'| = |V| - 1$).
2. **Fase di riscaldamento (Warm-up):** Ciascun algoritmo viene eseguito per 100 iterazioni consecutive non registrate. Questa procedura garantisce che le sezioni di codice più calde (*hot spots*) vengano intercettate dal profiler interno della JVM e tradotte direttamente in linguaggio macchina nativo altamente ottimizzato (loop unrolling, vettorizzazione hardware SIMD), stabilizzando i tempi prima della misurazione effettiva.
3. **Campionamento iterativo:** La misurazione reale viene calcolata estraendo la media aritmetica su un blocco di 50 generazioni distinte per ogni singola taglia spaziale, utilizzando la funzione nativa di precisione `System.nanoTime()` convertita successivamente in scala millisecondi (*ms/op*).

5.2 Risultati sperimentali globali

Le istanze di input seguono una progressione spaziale quadrata definita dall'insieme di nodi V , con un lato della griglia $N \in \{10, 20, 30, 50, 100\}$, coprendo un dominio di calcolo che spazia da un minimo di 100 celle fino a un massimo di 10.000 celle complessive per labirinto.

Nella Tabella seguente sono racchiusi i dati empirici ricavati dall'esecuzione del benchmark headless:

Algoritmo	10x10	20x20	30x30	50x50	100x100
Randomized DFS	0,0087	0,0234	0,0810	0,1817	0,6595
Randomized Kruskal	0,0198	0,0329	0,0787	0,2441	1,3635
Randomized Prim	0,0118	0,0250	0,0617	0,2144	0,8499
Aldous-Broder	0,0601	0,1593	0,3219	1,2879	7,3361
Wilson's Algorithm	0,0331	0,2088	0,7759	4,9501	82,3766
Recursive Division	0,0126	0,0160	0,0393	0,0897	0,3447
Eller's Algorithm	0,0148	0,0722	0,1292	0,3441	1,7317

Tabella 1: Tempi medi di esecuzione espressi in millisecondi (*ms/op*) al variare delle dimensioni della griglia.

5.3 Discussione critica dei risultati

L'analisi comparativa dei dati campionati mette in evidenza una perfetta e coerente corrispondenza tra la complessità asintotica teorica descritta nel Capitolo 2 e il comportamento macroscopico degli algoritmi sulla memoria fisica della Java Virtual Machine. I sette algoritmi testati rivelano dinamiche di scaling nettamente distinte, che permettono di classificarli in tre macro-regimi prestazionali ben definiti.

5.3.1 Il dominio degli approcci Top-Down: Divisione Ricorsiva

I dati sperimentali raccolti in fase di benchmark evidenziano come l'algoritmo di Divisione Ricorsiva registri in assoluto il tempo di esecuzione medio più basso sull'intera suite di test, fermando il cronometro a soli 0,3447 ms sulla griglia massima di dimensioni 100×100 .

Questo riscontro empirico trova una coerente giustificazione nella corretta formulazione della complessità asintotica temporale del paradigma. Come dimostrato mediante l'applicazione del Teorema dell'Esperto (Caso 1), l'algoritmo esibisce una complessità linearmente pura pari a $\mathcal{O}(V)$. Poiché il costo computazionale richiesto per l'erezione dei muri divisorii decresce geometricamente procedendo verso il basso nell'albero della ricorsione ($\mathcal{O}(\sqrt{V})$), il tempo di calcolo complessivo è interamente dominato e limitato dal numero di nodi foglia, ovvero dal volume totale delle celle.

Oltre alla natura lineare della complessità teorica, la fluidità microscopica registrata a runtime sulla macchina di test è determinata da un fattore micro-architetturale di fondamentale importanza: l'assenza di strutture dati ausiliarie complesse. L'algoritmo non necessita infatti di code di priorità, insiemi disgiunti o array dinamici di tracciamento del percorso. La scomposizione si appoggia esclusivamente sullo stack di sistema tramite record di attivazione snelli, riducendo al minimo l'impatto sui cicli di clock del processore.

La Divisione Ricorsiva si configura pertanto come il paradigma più efficiente in termini di *wall-clock time*, dimostrando come la sinergia tra una complessità lineare $\mathcal{O}(V)$ e una gestione minimale dell'impronta di memoria possa convertire l'astrazione matematica nella massima ottimizzazione hardware possibile.

5.3.2 L'efficienza dei Graph Traversal e delle strutture dati: DFS, Prim, Kruskal ed Eller

Nel secondo regime prestazionale si posizionano gli algoritmi operanti direttamente sulle strutture del grafo, caratterizzati da uno scaling lineare o quasi-lineare controllato e da tempi di calcolo compresi tra 0,65 ms e 1,73 ms sulla taglia 100×100 :

- **Randomized DFS (0,6595 ms):** Si conferma l'approccio orientato ai nodi più rapido. Operando in tempo lineare $\mathcal{O}(V)$, l'uso di uno stack strutturato riduce l'overhead di allocazione al minimo. La progressione rigorosamente monotonica e stabile dei dati (0,0810 ms $\rightarrow$ 0,1817 ms $\rightarrow$ 0,6595 ms) convalida l'efficacia della fase di *warm-up* a 100 iterazioni, la quale ha indotto la compilazione nativa JIT (livello C2) in uno stato stazionario stabile prima della raccolta dei dati, eliminando fluttuazioni o ritardi parassiti a runtime.
- **Randomized Prim Modificato (0,8499 ms):** Esibisce uno scaling eccellente, posizionandosi immediatamente a ridosso della DFS. Questo successo metrologico è determinato dall'integrazione dello *swap-to-last trick* in fase di rimozione dei nodi dalla frontiera. Sostituendo il metodo `remove()` standard di `ArrayList` (che comporterebbe uno slittamento lineare degli elementi in memoria ad ogni estrazione), la tecnica scambia l'elemento selezionato con l'ultimo elemento del vettore prima di effettuarne l'eliminazione. Il costo di estrazione e di aggiornamento dinamico della frontiera viene così ridotto a tempo costante puro $\mathcal{O}(1)$, permettendo a Prim di riflettere fedelmente la complessità lineare teorica $\mathcal{O}(V)$ e di staccare l'algoritmo di Kruskal.

- **Randomized Kruskal (1,3635 ms):** Mostra uno scaling controllato grazie all'efficienza ammortizzata della struttura dati *Disjoint-Set* (Union-Find) dotata di compressione dei cammini e bilanciamento per rango. L'overhead principale che lo distanzia da Prim e DFS è localizzato nella fase di inizializzazione stocastica, in cui il metodo `Collections.shuffle` deve mescolare linearmente in memoria un vettore ad alta densità contenente la totalità degli archi della griglia prima di avviare il ciclo di unione.
- **Eller's Algorithm (1,7317 ms):** Pur registrando la latenza più alta di questo comparto, l'algoritmo mantiene un andamento strettamente lineare. L'overhead residuo rispetto a Kruskal non è legato alla logica dei set, ma alla gestione dinamica delle tabelle associative di hashing (`java.util.Map`) necessarie per raggruppare, dividere e riassegnare i set dei cammini durante la complessa proiezione vettoriale eseguita riga per riga.

5.3.3 Analisi comparativa dei paradigmi stocastici: Wilson e Aldous-Broder

La terza e ultima macro-categoria del benchmark è rappresentata dai paradigmi basati su passeggiate casuali (*Random Walks*): **Aldous-Broder** (7,3361 ms) e **Wilson** (82,3766 ms).

In questo framework, l'algoritmo di Wilson è stato ottimizzato introducendo una matrice booleana di supporto (`inWalk`) per il tracciamento istantaneo della traiettoria corrente. Questa scelta architetturale consente di verificare la formazione di un ciclo in tempo costante puro $\mathcal{O}(1)$ superando l'anti-pattern computazionale legato alla ricerca lineare sequenziale di un metodo ingenuo come `ArrayList.indexOf()`. Di conseguenza, l'estrazione del cammino depurato dai cicli (Loop-Erased Random Walk) avviene in modo efficiente, rispecchiando la complessità teorica intrinseca del paradigma che, su griglie bidimensionali, è asintoticamente pari a $\mathcal{O}(V \log^2 V)$.

Nonostante l'ottimizzazione della cancellazione dei cicli, l'algoritmo di Wilson fa registrare un tempo pari a 82,3766 ms sulla taglia massima di 10.000 celle, posizionandosi al di sopra di Aldous-Broder (7,3361 ms). Tale discrepanza nel *wall-clock time* reale è determinata da un fattore amministrativo intrinseco alla logica dei due algoritmi:

1. **L'overhead di selezione della radice:** Ogni volta che una passeggiata casuale di Wilson converge unendosi al labirinto, l'algoritmo deve individuare una nuova cella di partenza esterna. Nel codice, il metodo `getRandomUnvisitedCell` effettua una scansione lineare dell'intera griglia per collezionare i nodi non ancora visitati. Questo campionamento globale introduce un costo di gestione computazionale che grava sul tempo fisico complessivo.
2. **La linearità locale di Aldous-Broder:** Al contrario, l'algoritmo di Aldous-Broder esibisce una complessità teorica superiore ($\mathcal{O}(V \log^2 V)$) a causa del paradosso del collezionista di coupon, il quale costringe la passeggiata a muoversi a vuoto nelle fasi finali della generazione. Tuttavia, ogni singolo passo della sua passeggiata casuale viene eseguito localmente in tempo costante $\mathcal{O}(1)$, verificando semplicemente un flag booleano sulla cella adiacente. All'interno dei confini metrici testati (fino a 100×100), l'assenza strutturale di routine di scansione globale per

la selezione di nuove radici consente ad Aldous-Broder di mantenere un profilo di esecuzione compatto, superando l'algoritmo di Wilson nel mondo reale della JVM.

Riferimenti bibliografici

- [1] Wikipedia contributors, “Maze generation algorithm,” *Wikipedia, The Free Encyclopedia*, https://en.wikipedia.org/wiki/Maze_generation_algorithm.